

Tema 9: Implementación de vectores

1. Estructura interna y Atributos

Un vector es un **array dinámico** en el *heap*. Para gestionarlo, usamos un **namespace** propio y tres atributos:

- **T* data_**: Puntero al bloque de memoria donde guardamos los elementos.
- **int size_**: Elementos ocupados actualmente.
- **int capacity_**: Memoria total reservada.

```
namespace pro2 {  
    template <typename T>  
    class Vector {  
        T* data_;  
        int size_, capacity_;  
        void reallocate_(int new_cap); // La "mudanza"  
    public:  
        using iterator = T*; // Iterador simple = Puntero  
        // ... métodos ...  
    };  
}
```

2. Regla de los tres (Gestión de memoria)

Si una clase gestiona memoria dinámica, necesita estos 3 métodos para evitar **segfaults** o **leaks**:

A. Constructor de copia (Deep Copy)

Crea una copia real en un nuevo bloque de memoria, no solo copia el puntero.

```
Vector(const Vector& v) {  
    data_ = new T[v.capacity_];  
    size_ = v.size_;  
    capacity_ = v.capacity_;  
    for (int i = 0; i < size_; ++i) data_[i] = v.data_[i];  
}
```

B. Operador de asignación

Libera la memoria vieja antes de copiar la nueva.

```
Vector& operator=(const Vector& v) {  
    if (this != &v) {  
        delete[] data_; // 1. Limpieza  
        data_ = new T[v.capacity_]; // 2. Reserva  
        size_ = v.size_;  
        capacity_ = v.capacity_;  
        for (int i = 0; i < size_; ++i) data_[i] = v.data_[i]; // 3. Copia  
    }  
}
```

```

    return *this;
}

```

C. Destructor

El único que libera la memoria definitivamente.

```

~Vector() { delete[] data_; }

```

3. Operadores

Categoría	Definición de la función (cpp)	Llamada desde el main
Acceso	<code>T& operator[](int i)</code>	<code>v[i] = x;</code>
Comparación	<code>bool operator<(const V& v)</code>	<code>if (a < b)</code>
Asignación	<code>void operator+=(const V& v)</code>	<code>a += b;</code>
Aritméticos	<code>V operator+(const V& v)</code>	<code>c = a + b;</code>
Flujo	<code>ostream& operator<<(ostream& o, const V& v)</code>	<code>cout << v;</code>

```

Stack<int> s1, s2;
s1 += s2;           // Sintaxis natural (en el main)
s1.operator+=(s2);  // Equivalente a s1 += s2;

```

Cuando escribes `s += s2` en el main, el compilador de C++ busca si la clase tiene definida una función que se llame literalmente `operator+=`. Si la encuentra y acepta los argumentos que le pasas (en este caso un objeto de tipo `Stack`), realiza la llamada directa.

4. Acceso e iteradores

El acceso es $\Theta(1)$ directo por aritmética de punteros. - **El operador []**: Se duplica para permitir lectura en objetos `const`. - **Iteradores**: En un vector, un `iterator` es simplemente un `T*`.

```

T& operator[](int i) { return data_[i]; }
const T& operator[](int i) const { return data_[i]; }

```

```

iterator begin() { return data_; }
iterator end() { return data_ + size_; }

```

5. El motor: `reallocate_` (La mudanza)

Método privado que cambia la capacidad del vector. Operación costosa $\mathcal{O}(n)$. 1. Pide un nuevo bloque. 2. Copia los elementos viejos. 3. `delete[]` bloque viejo. 4. Actualiza puntero y capacidad.

```

void reallocate_(int new_cap) {
    T* new_data = new T[new_cap];           // 1. Nuevo bloque
    for (int i = 0; i < size_; ++i) {
        new_data[i] = data_[i];             // 2. Copia
    }
    delete[] data_;                          // 3. Limpieza viejo
    data_ = new_data;                        // 4. Actualiza
    capacity_ = new_cap;
}

```

6. Inserción y crecimiento

- **push_back**: Si está lleno, **dobla** la capacidad.
- **Costo Amortizado**: Aunque redimensionar es $\mathcal{O}(n)$, solo ocurre cada 2^k veces. El promedio es $\mathcal{O}(1)$.

```
void push_back(const T& x) {
    if (size_ == capacity_) {
        reallocate_(capacity_ == 0 ? 1 : 2 * capacity_);
    }
    data_[size_++] = x;
}

::vectorviz
```

7. Extracción y Thrashing

Para evitar oscilaciones costosas (doblar/reducir constantemente): - **Estrategia**: Esperar a estar a **1/4** de capacidad para reducirla a la **mitad**.

```
void pop_back() {
    --size_;
    if (size_ > 0 && size_ == capacity_ / 4) {
        reallocate_(capacity_ / 2);
    }
}
```

Resumen de Complejidad

Operación	Complejidad	Nota
Acceso [i]	$\Theta(1)$	Directo.
push_back	$\mathcal{O}(1)^*$	*Amortizado. Peor caso $\mathcal{O}(n)$.
pop_back	$\mathcal{O}(1)^*$	*Amortizado. Evitamos el Thrashing.
insert/erase	$\mathcal{O}(n)$	Hay que desplazar todos los elementos posteriores.
size/empty	$\Theta(1)$	Consulta de atributos.

Extra: Operador de salida

Muy útil para depurar la implementación:

```
template <typename T>
ostream& operator<<(ostream& os, const Vector<T>& v) {
    os << "[";
    for (int i = 0; i < v.size(); ++i) {
        os << v[i] << (i == v.size() - 1 ? "" : ",");
    }
    return os << "]";
}
```